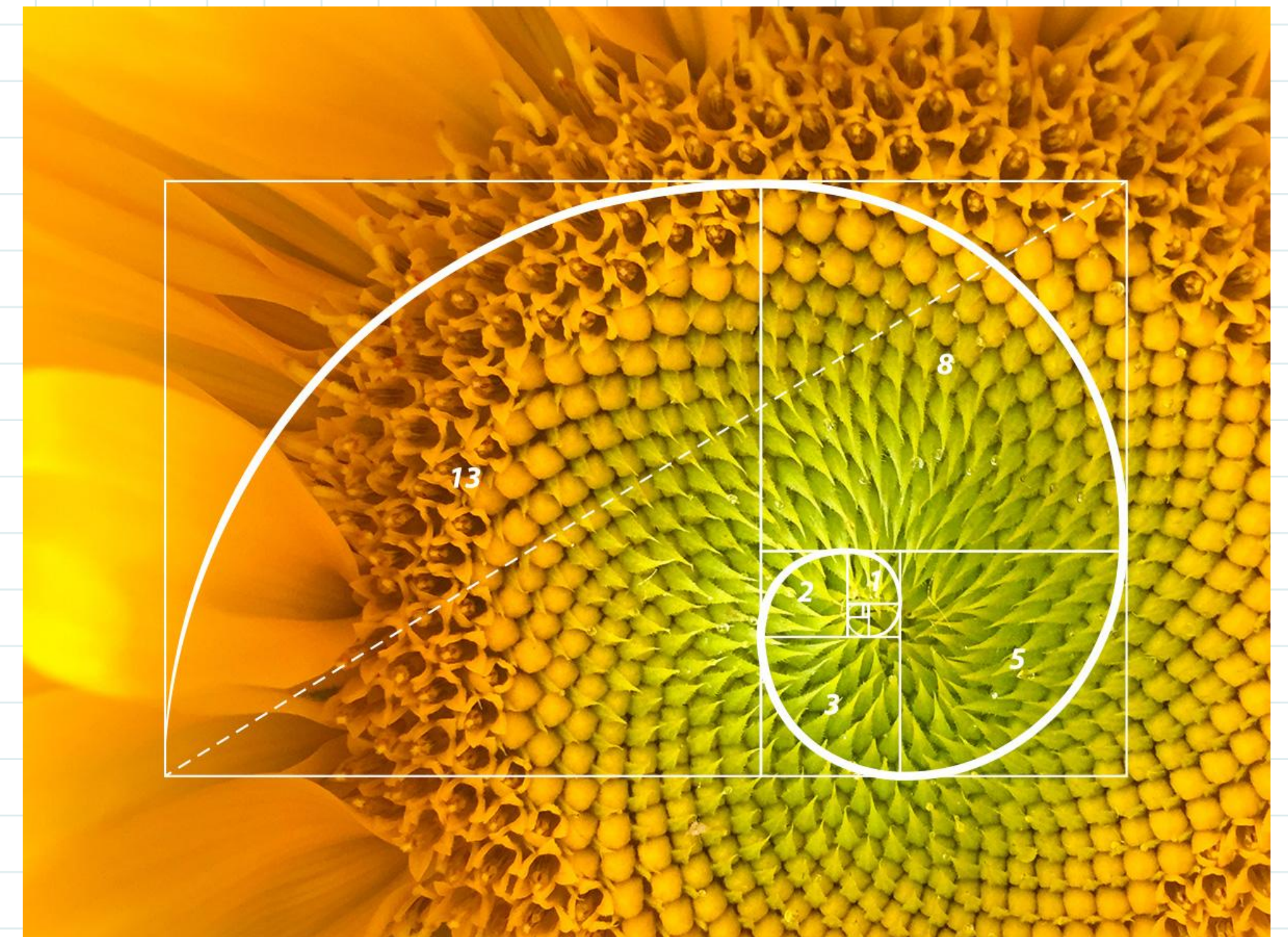


MACHINE LEARNING

Principal Component Analysis (PCA)

Dr. Sirojul Munir, S.Si., M.Kom.
rojulman@nurulfikri.ac.id

ARTIFICIAL INTELLIGENCE – INFORMATICS STTNF



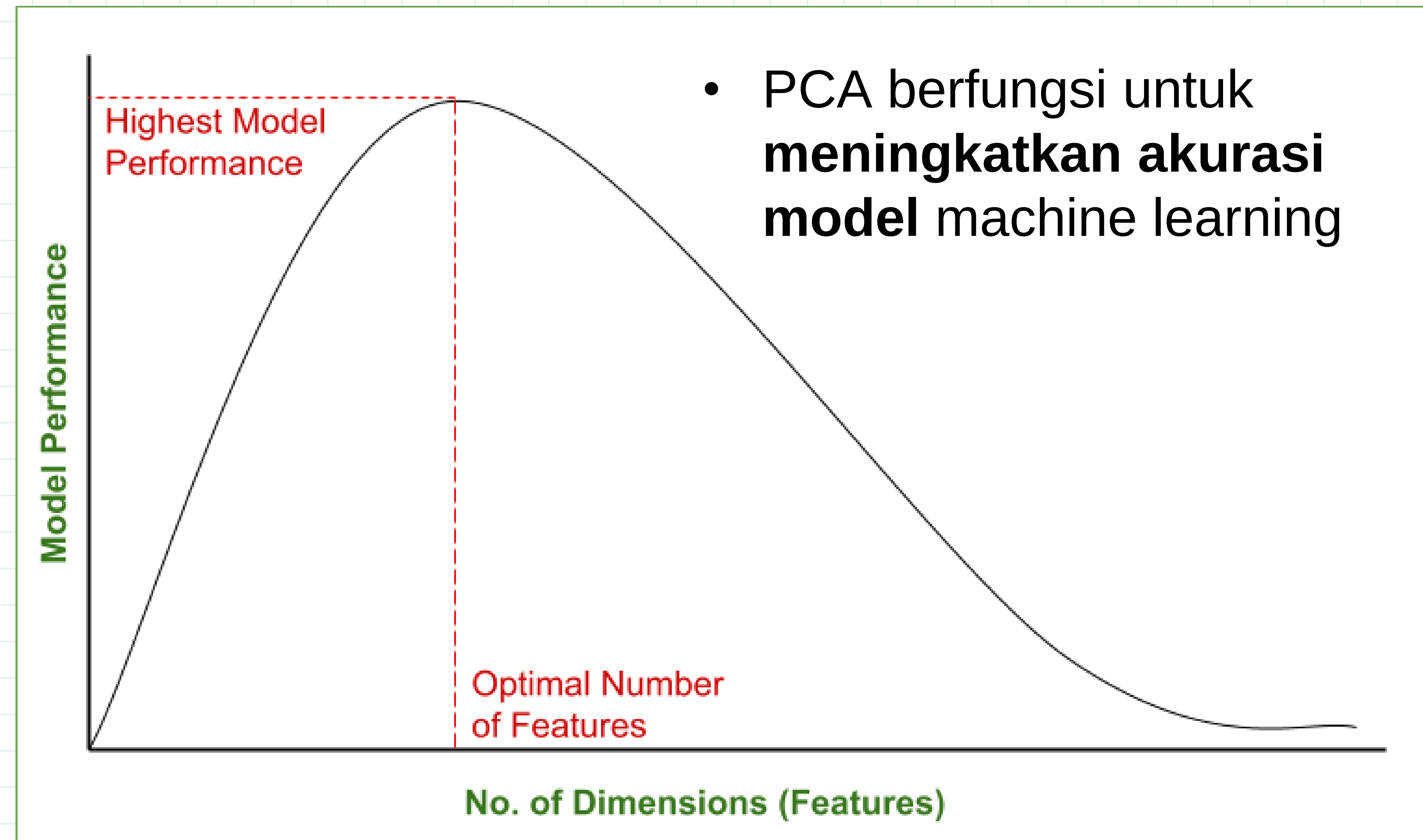
Daur ulang Project Data Science



PCA – Principal Component Analysis

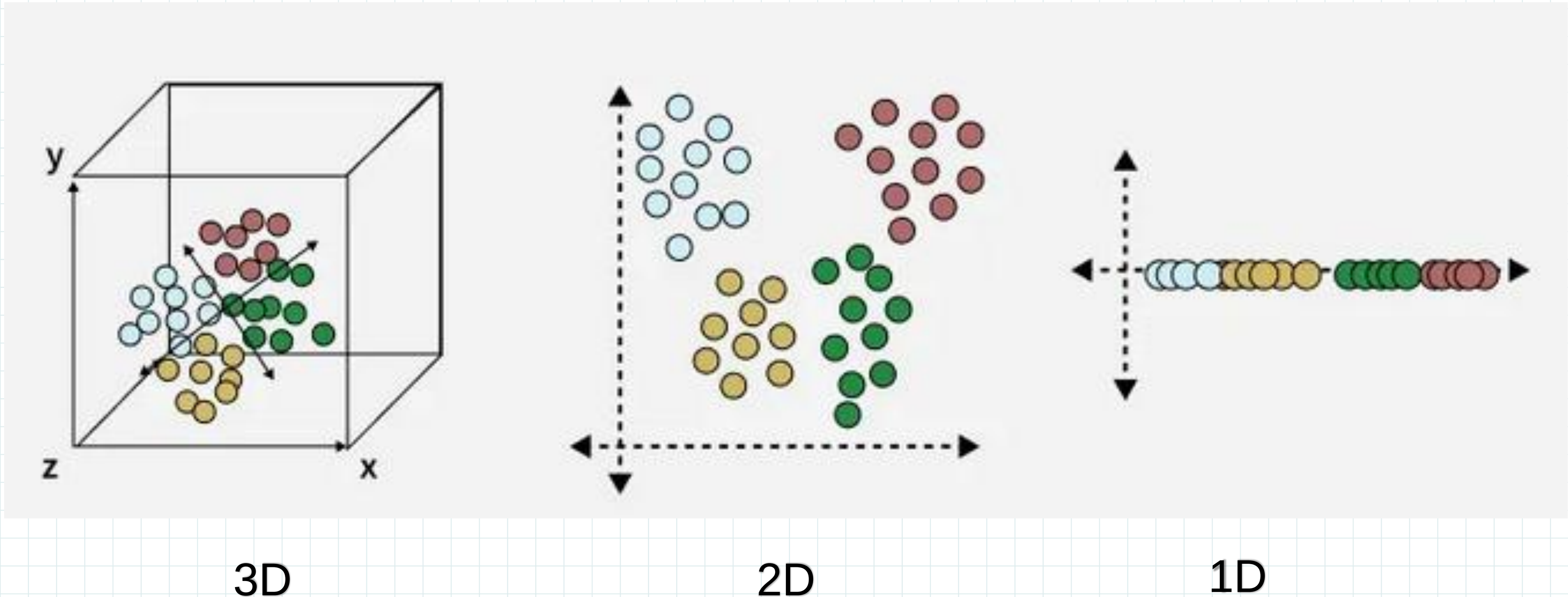
- Metode mendapatkan fitur penting dari suatu dataset
- Bekerja dengan cara mencari variasi terbesar dari dataset
- Menemukan fitur baru yang tidak berkorelasi dengan fitur lain disebut dengan PC (Principal Component) atau Komponen Utama

Performa Model vs Dimensi Fitur



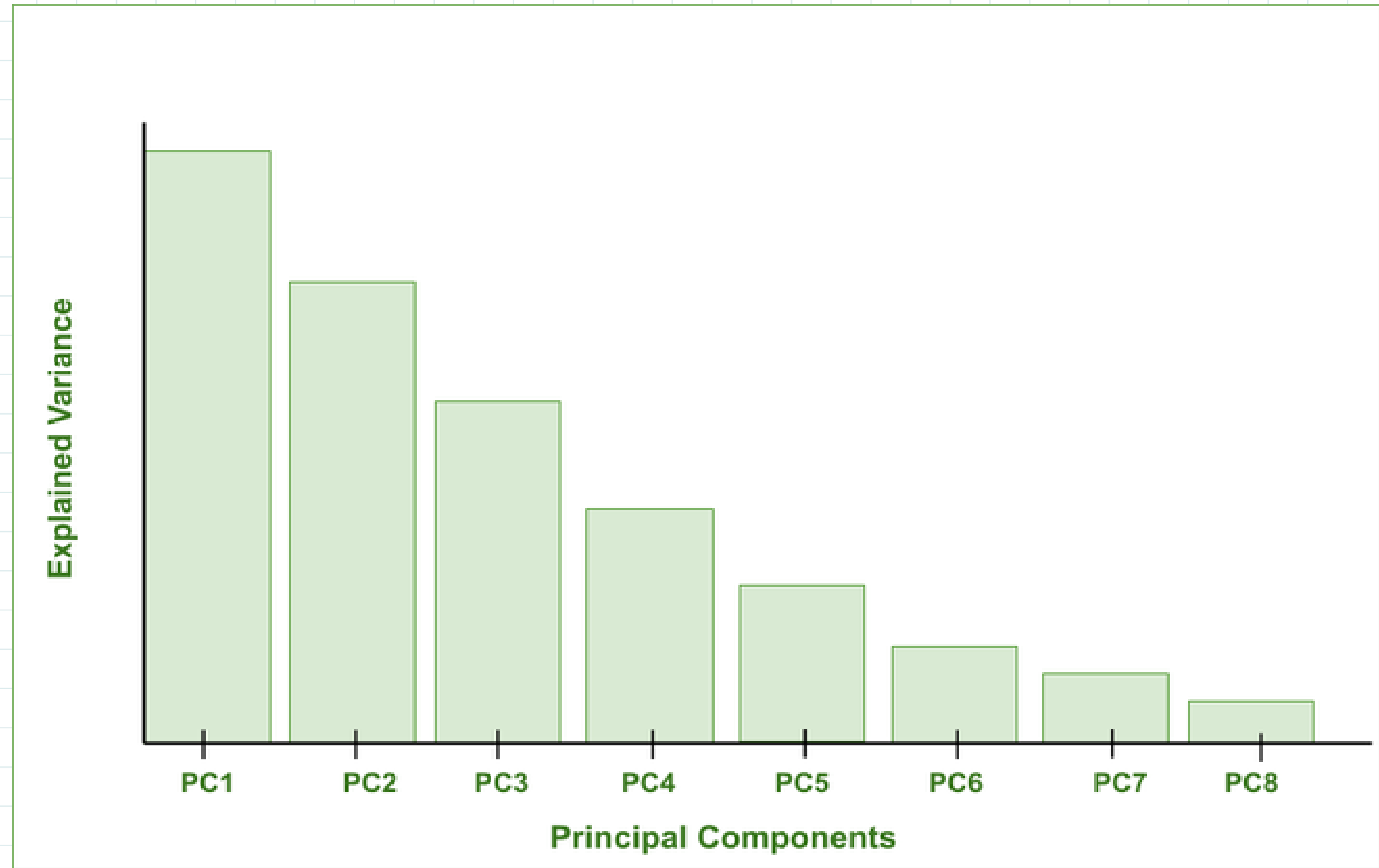
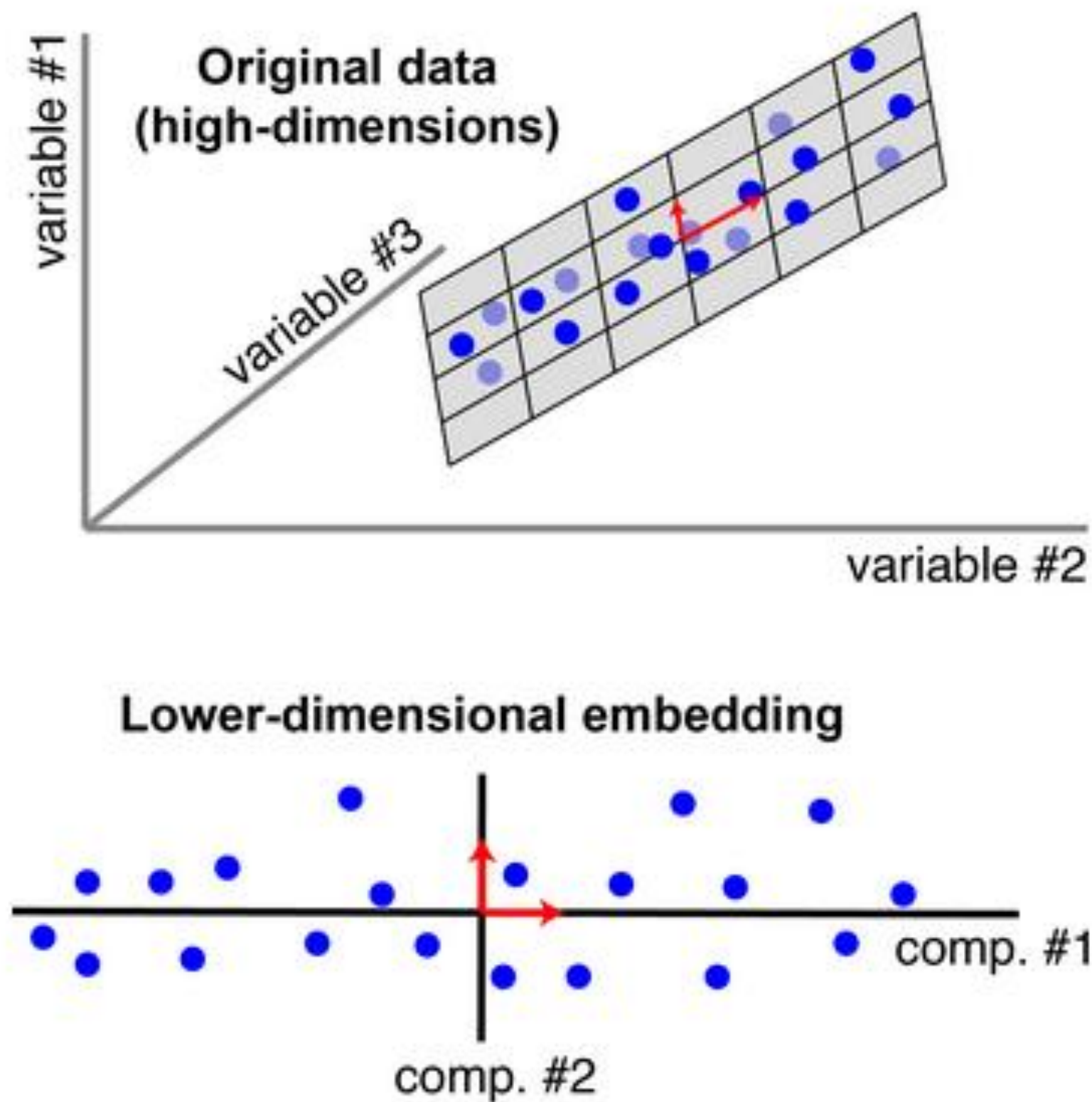
PCA – Principal Component Analysis

- PCA **mereduksi** dimensi data tinggi menjadi hanya beberapa variabel dengan mengambil nilai maksimum dari variasi dataset



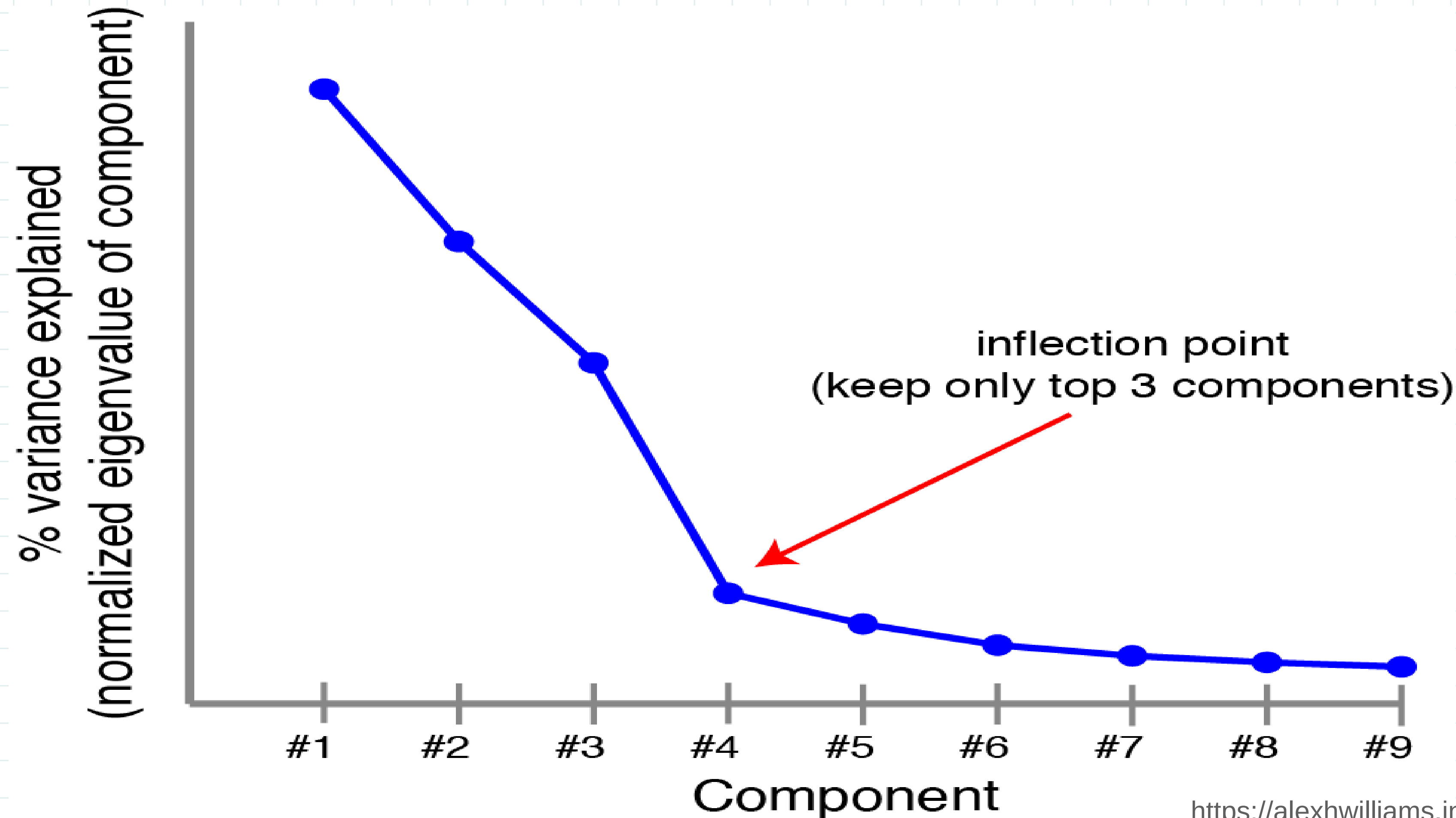
PCA – Principal Component Analysis

- PCA **mereduksi** dimensi data tinggi menjadi hanya beberapa variabel dengan mengambil nilai maksimum dari variasi dataset



PCA – Principal Component Analysis

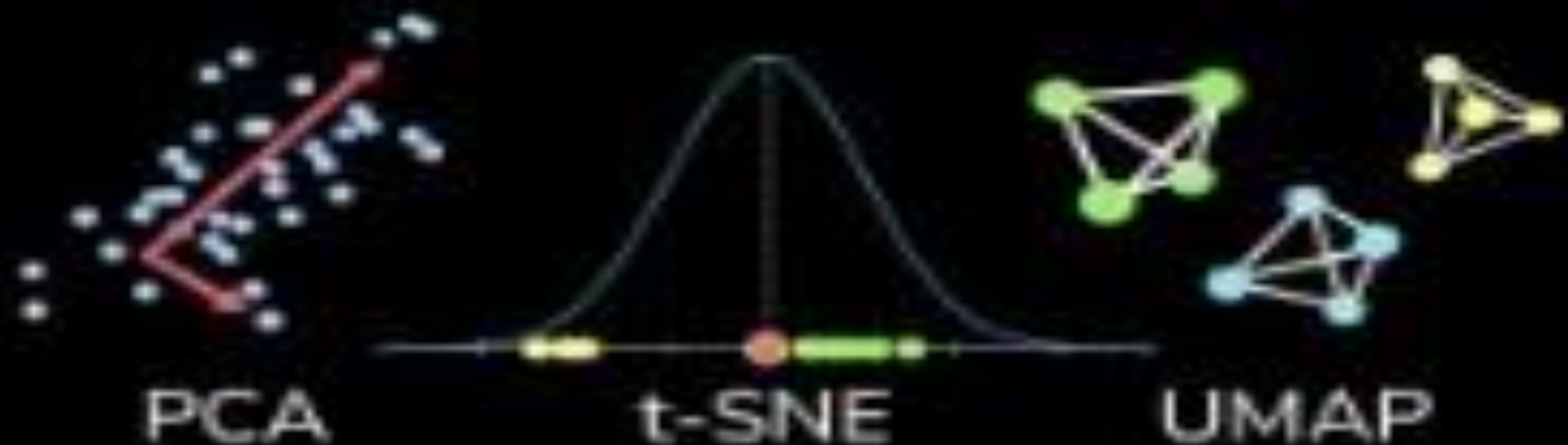
- PCA **mereduksi** dimensi data tinggi menjadi hanya beberapa variabel dengan mengambil nilai maksimum dari variasi dataset



<https://alexhwilliams.info/itsneuronalblog/2016/03/27/pca/>

PCA – Principal Component Analysis

Latent Space Visualisation



https://www.youtube.com/watch?v=o_cAOa5fMhE

PCA – Formula Matematika

1. Standardisasi fitur:

$$z_{ij} = (x_{ij} - \mu_j) / \sigma_j$$

μ_j : rata-rata fitur j

σ_j : standar deviasi fitur j

2. Matrix Kovarian C

$$C = (1 / (n - 1)) \cdot Z^T Z$$

3. Eigen Decomposition

$$C v_k = \lambda_k v_k$$

v_k : eigenvector (principal component ke-k)

λ_k : eigenvalue (varian yang dijelaskan)

4. Proyeksi Data ke Ruang PCA

$$Y = Z V_k$$

V_k : matriks eigenvector utama

Y: data hasil proyeksi

Penerapan PCA

1. Data Visualization

PCA mereduksi data berdimensi tinggi ke dalam ruang berdimensi dua / tiga, agar visualisasinya lebih mudah, serta membantu mengungkap tren data, kluster, atau outlier

2. Preprocessing for Machine Learning

PCA sering digunakan sebelum menerapkan algoritma pembelajaran mesin (pra-proses data) untuk mengurangi dimensionalitas, mempercepat pelatihan, dan meningkatkan generalisasi model.

3. Feature Extraction

PCA mengubah fitur data berkorelasi menjadi serangkaian komponen tidak berkorelasi yang lebih kecil, yang dapat membantu meningkatkan kinerja dan interpretabilitas model.

Penerapan PCA

4. Data Compression

PCA mengurangi kebutuhan penyimpanan data dengan merepresentasikan data dengan **dimensi** yang lebih **sedikit** sambil **mempertahankan** sebagian besar **variabilitasnya**, membuatnya **efektif** dalam **kompresi gambar dan sinyal**.

5. Noise Reduction

Dengan menyaring komponen varians rendah, PCA dapat **mengurangi gangguan** dalam **data** untuk aplikasi seperti **gambar dan audio**.

PCA – Iris Dataset

- Step 1: Load Data

```
[1]: import pandas as pd

# Read the CSV file with a comma delimiter
df = pd.read_csv('../data/iris/iris.csv', sep=',')

# cetak header data (5 baris data) dari file
df.head()
```

Features / Attributes / Variables

Samples

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

PCA – Iris Dataset

- Step 2: Standarisasi Struktur Data Fitur

```
from sklearn.preprocessing import StandardScaler

features = ['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']

# Separating out the features
x = df.loc[:, features].values

# Separating out the target
y = df.loc[:, ['Species']].values

# Standardizing the features
x = StandardScaler().fit_transform(x)
# print head

df_standar = pd.DataFrame(x, columns=features)
df_standar.head()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa



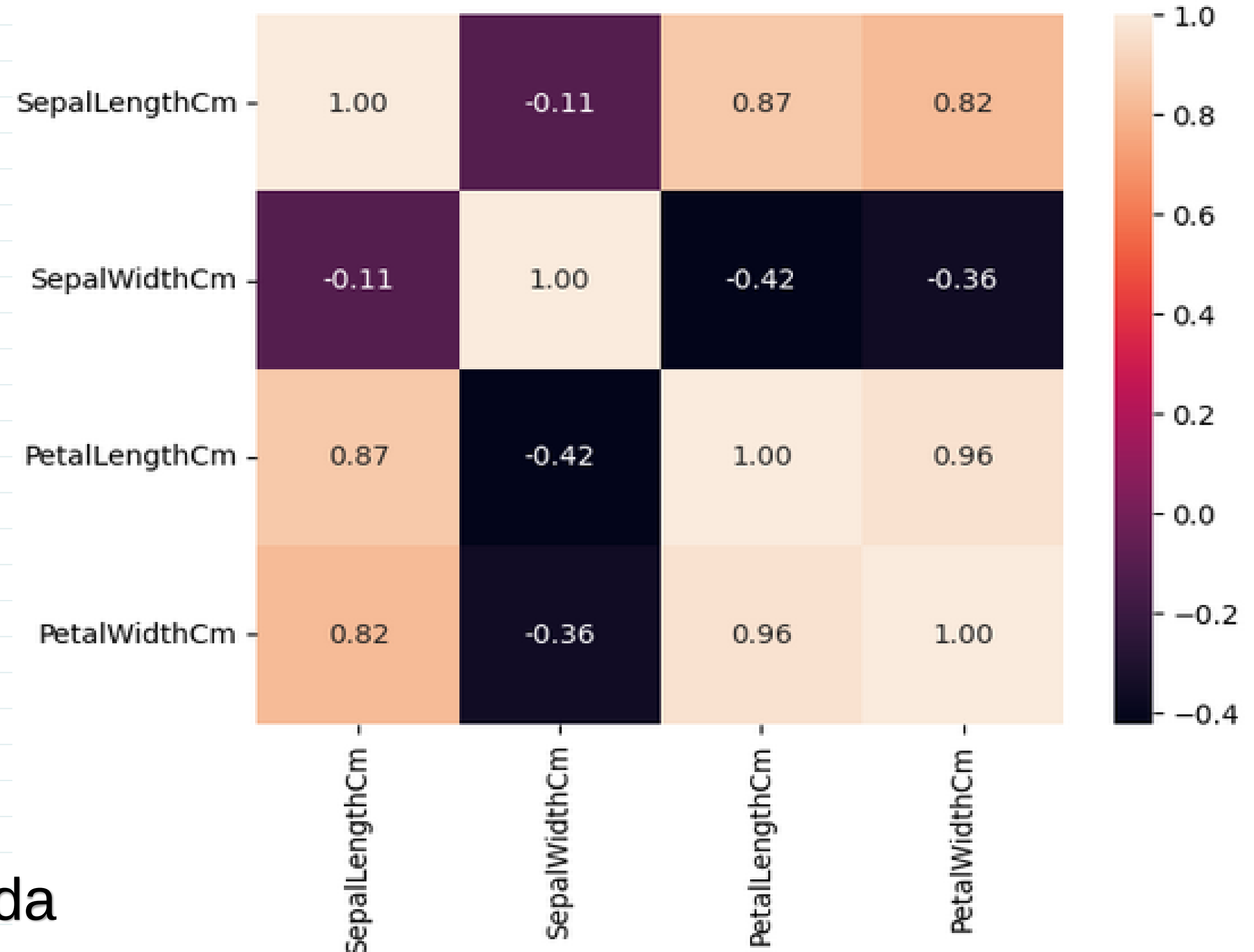
	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
0	-0.900681	1.032057	-1.341272	-1.312977
1	-1.143017	-0.124958	-1.341272	-1.312977
2	-1.385353	0.337848	-1.398138	-1.312977
3	-1.506521	0.106445	-1.284407	-1.312977
4	-1.021849	1.263460	-1.341272	-1.312977

PCA – Iris Dataset

- Cek Koefesien Korelasi dari Fitur tanpa PCA
- Gunakan HeatMap Analysis

```
import seaborn as sns
sns.heatmap(df_standar.corr(),
            annot=True, fmt=".2f")
```

Analisis apakah diperlukan reduksi fitur ?
Korelasi antar fitur masih ada



PCA – Iris Dataset

- Step 3: Terapkan PCA

```
from sklearn.decomposition import PCA
#Applying PCA
#Taking no. of Principal Components as 3
pca = PCA(n_components = 3)
pca.fit(df_standar)
data_pca = pca.transform(df_standar)
data_pca = pd.DataFrame(data_pca, columns=['PC1', 'PC2', 'PC3'])
data_pca.head()
```

```
pca.explained_variance_ratio_
```

```
array([0.72770452, 0.23030523, 0.03683832])
```

99,3%

Komponen Fitur Asli Terwakili

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
0	-0.900681	1.032057	-1.341272	-1.312977
1	-1.143017	-0.124958	-1.341272	-1.312977
2	-1.385353	0.337848	-1.398138	-1.312977
3	-1.506521	0.106445	-1.284407	-1.312977
4	-1.021849	1.263460	-1.341272	-1.312977

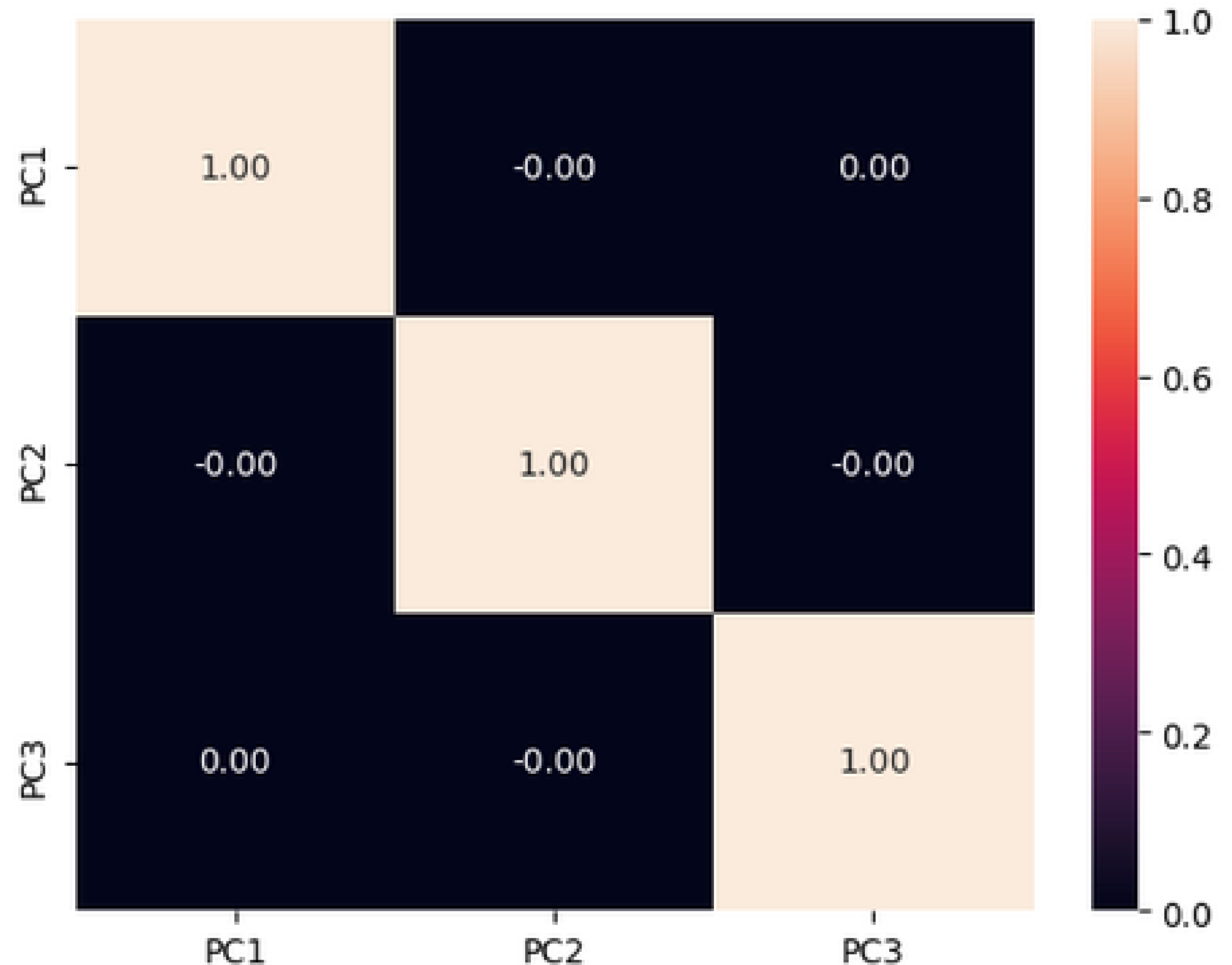


	PC1	PC2	PC3
0	-2.264542	0.505704	0.121943
1	-2.086426	-0.655405	0.227251
2	-2.367950	-0.318477	-0.051480
3	-2.304197	-0.575368	-0.098860
4	-2.388777	0.674767	-0.021428

PCA – Iris Dataset

- Cek Koefesien Korelasi PCA
- Gunakan HeatMap Analysis

```
## cek heatmap kembali untuk fitur PCA
sns.heatmap(data_pca.corr(),
            annot=True, fmt=".2f")
```



Keterkaitan / korelasi antara komponen fitur PCA minim (nihil)

PCA – Iris Dataset

- Grafik 3D antara komponen PCA #3 dengan target data

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

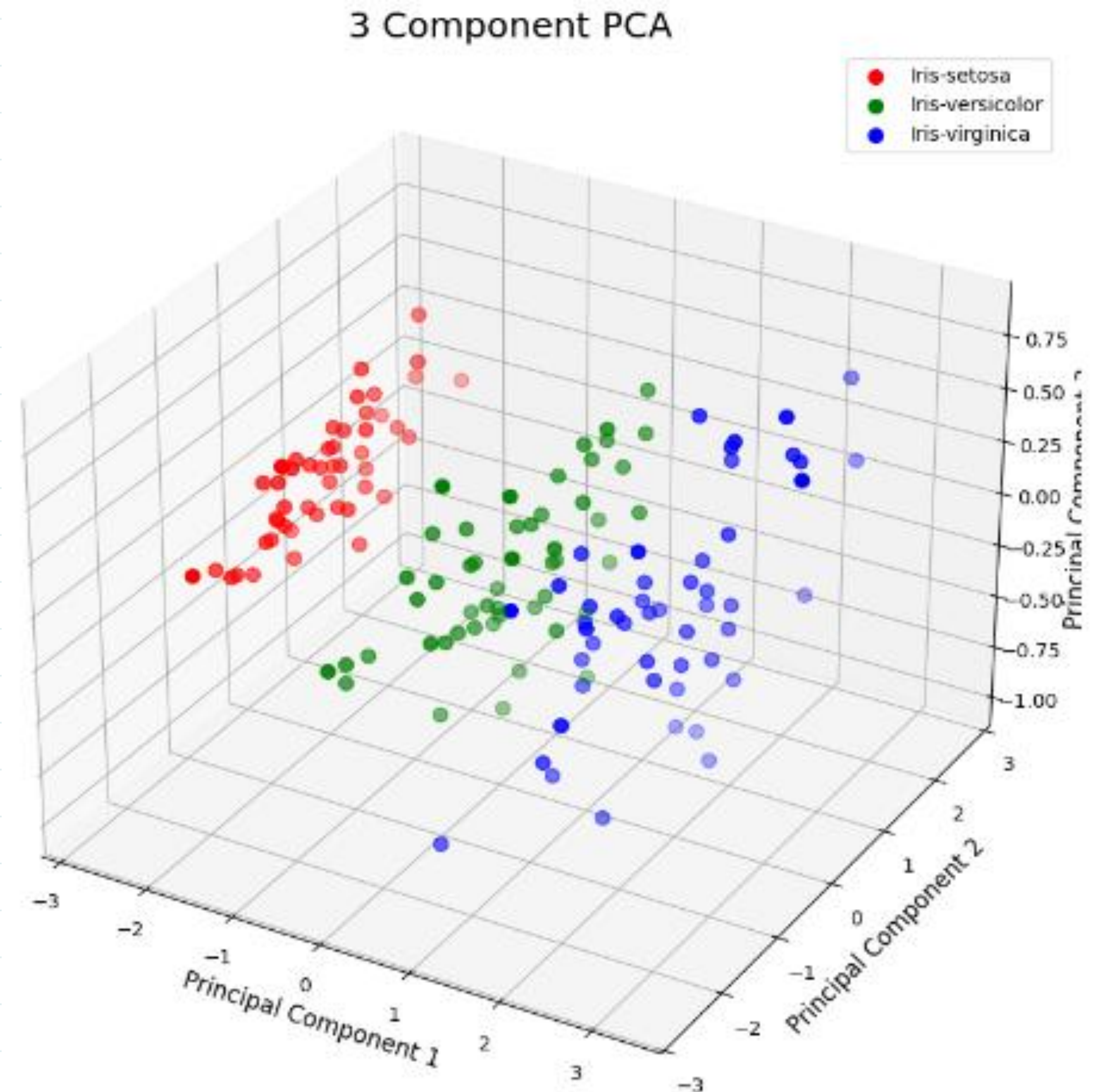
fig = plt.figure(figsize=(12,10))
ax = fig.add_subplot(111, projection='3d')

ax.set_xlabel('Principal Component 1', fontsize=12)
ax.set_ylabel('Principal Component 2', fontsize=12)
ax.set_zlabel('Principal Component 3', fontsize=12)
ax.set_title('3 Component PCA', fontsize=18)

targets = ['Iris-setosa', 'Iris-versicolor', 'Iris-virginica']
colors = ['r', 'g', 'b']

for target, color in zip(targets, colors):
    indicesToKeep = finalDf['Species'] == target
    ax.scatter(
        finalDf.loc[indicesToKeep, 'PC1'],
        finalDf.loc[indicesToKeep, 'PC2'],
        finalDf.loc[indicesToKeep, 'PC3'],
        c=color,
        s=50,
        label=target
    )

ax.legend()
plt.show()
```



PCA – Iris Dataset

- Grafik 3D antara komponen PCA #2 dengan target data

```
import matplotlib.pyplot as plt

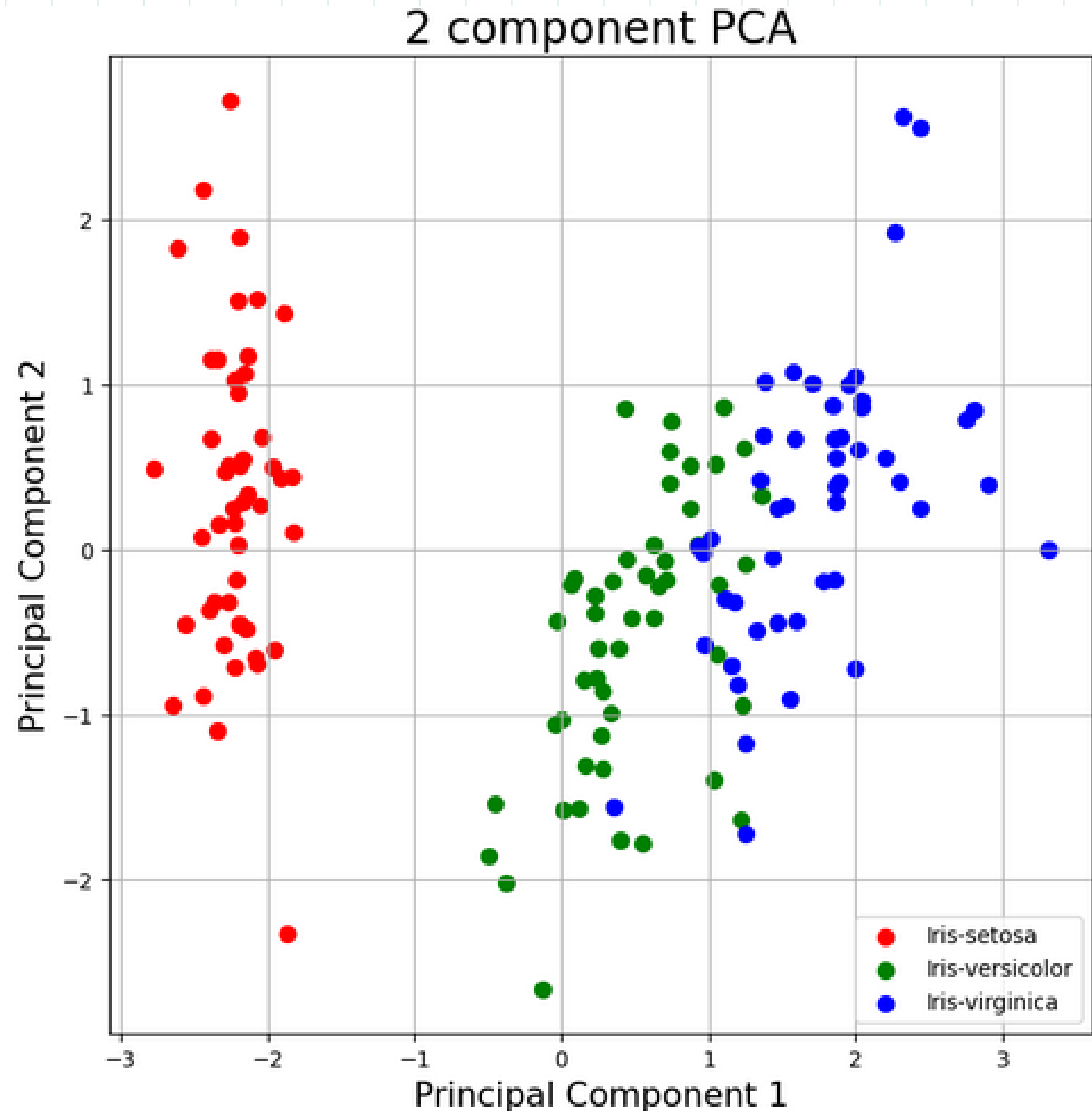
pca = PCA(n_components = 2)
pca.fit(df_standar)
data_pca = pca.transform(df_standar)
data_pca = pd.DataFrame(data_pca, columns=['PC1', 'PC2'])
#data_pca.head()

finalDf = pd.concat([data_pca, df[['Species']]], axis = 1)
fig = plt.figure(figsize = (8,8))
ax = fig.add_subplot(1,1,1)
ax.set_xlabel('Principal Component 1', fontsize = 15)
ax.set_ylabel('Principal Component 2', fontsize = 15)
ax.set_title('2 component PCA', fontsize = 20)

targets = ['Iris-setosa', 'Iris-versicolor', 'Iris-virginica']
colors = ['r', 'g', 'b']
for target, color in zip(targets, colors):
    indicesToKeep = finalDf['Species'] == target
    ax.scatter(finalDf.loc[indicesToKeep, 'PC1'],
              finalDf.loc[indicesToKeep, 'PC2'],
              c = color,
              s = 50)
ax.legend(targets)
ax.grid()

print(pca.explained_variance_ratio_)
```

[0.72770452 0.23030523] ➡ 95,7%



Fitur PCA – Pemodelan Klasifikasi SVM

```
## from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris

# 1. Load dataset Iris
df = pd.read_csv('../data/iris/iris.csv', sep=',')

# Separating out the features
X = df.loc[:, features].values

# Separating out the target
y = df.loc[:, ['Species']].values
#X = iris.data
#y = iris.target

# 2. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# 3. Standarisasi data
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # fit + transform
X_test_scaled = scaler.transform(X_test) # hanya transform

# 4. PCA - ambil 2 komponen utama
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

# 5. SVM model (RBF kernel)
svm_model = SVC(kernel='rbf', gamma='scale')
svm_model.fit(X_train_pca, y_train)

# 6. Prediksi
y_pred = svm_model.predict(X_test_pca)

# 7. Akurasi
acc = accuracy_score(y_test, y_pred) * 100.0
print("Akurasi SVM dengan PCA:", acc, "%")
```

Akurasi SVM dengan PCA: 90.0 %



Terima Kasih

<http://youtube.com/@rojulman>